MODULE 2 · CORE WORKLOADS

Multi-Container Pods

Sidecars, init containers, and the fabric they share

WHY IT MATTERS

One container isn't always enough

Real workloads need helper work that shouldn't be baked into the app image: a log shipper reading the app's output, a proxy sitting in front of it, a setup step that must finish before the app can start.

Kubernetes lets you compose that inside **one Pod** instead of one container — the helpers share the app's network and volumes without ever touching its filesystem or image.

BY THE END YOU CAN

- Explain sidecar vs init container
- Share data between containers via a volume
- Target one container with `-c`
- Read the init-then-app startup order

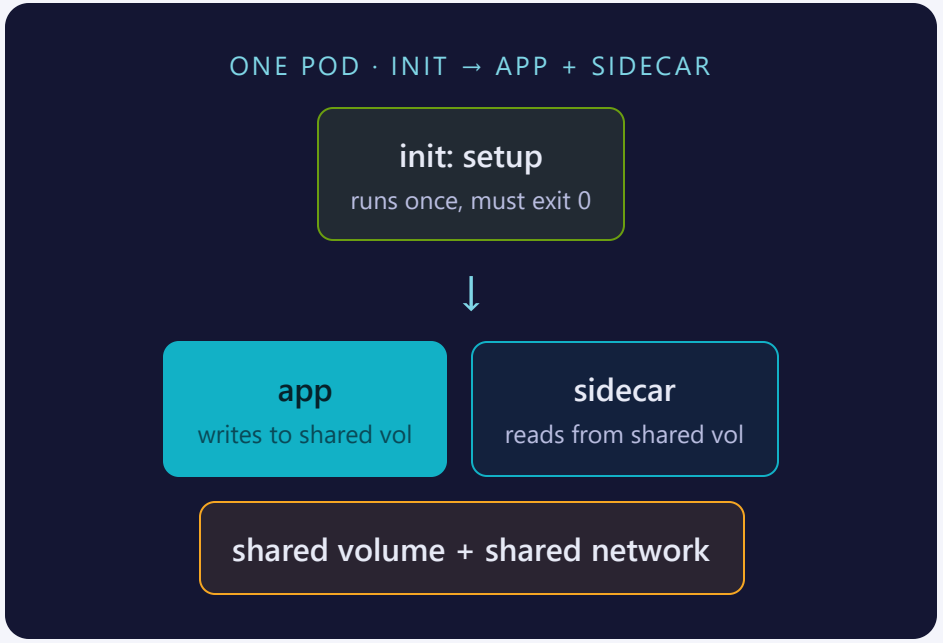
CONCEPT

Init runs to completion, then app + sidecar run together

A Pod can list two kinds of containers:

- **initContainers** — run one at a time, in order, and must each exit `0` before the next (or the app) starts
- **containers** — the app and any sidecars, started together and kept running for the Pod's life

All of them can mount the same volume — that's the handoff point between init, app, and sidecar.



SIDECAR VS INIT — THE KEY DIFFERENCE

Two patterns, opposite timing

	INIT CONTAINER	SIDECAR (APP) CONTAINER
When it runs	Before app containers, to completion	Alongside app containers, for the Pod's life
Order	Sequential, in listed order	Concurrent
Counts toward READY	No — must finish first	Yes — must stay running
Typical use	Fetch config, run migrations, wait for a dependency	Log shipping, proxy, metrics export

SINCE KUBERNETES 1.28

A **native sidecar** is an init container with `restartPolicy: Always` — starts before the app but keeps running alongside it. This lab uses the classic, version-portable patterns above.

MANIFEST ANATOMY

pod-init.yaml — the init container runs first

```yaml
# pod-init.yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
  namespace: training
  labels:
    app: init-demo
spec:
  volumes:
    - name: work
      emptyDir: {}
  # init container runs to completion BEFORE any app container starts
  initContainers:
    - name: setup
      image: busybox:1.36
      command:
        - sh
        - -c
        - "echo 'prepared by the init container' > /work/index.html"
      volumeMounts:
        - name: work
          mountPath: /work
  containers:
    # app container serves the file the init container prepared
    - name: web
      image: nginx:1.25
      volumeMounts:
        - name: work
          mountPath: /usr/share/nginx/html
      ports:
        - containerPort: 80
```

initContainers run once and must exit `0` before the **containers** list starts, handing off through the Pod-level `volumes` entry.

MANIFEST ANATOMY

pod-sidecar.yaml — app and sidecar share a volume

```yaml
# pod-sidecar.yaml
apiVersion: v1
kind: Pod
metadata:
  name: sidecar-demo
  namespace: training
  labels:
    app: sidecar-demo
spec:
  volumes:
    - name: shared
      emptyDir: {}
  containers:
    # main container: writes a timestamp into the shared volume every 5s
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - "while true; do date >> /shared/log.txt; sleep 5; done"
      volumeMounts:
        - name: shared
          mountPath: /shared
    # sidecar container: tails the same file from the shared volume
    - name: sidecar
      image: busybox:1.36
      command:
        - sh
        - -c
        - "sleep 6; tail -f /shared/log.txt"
      volumeMounts:
        - name: shared
          mountPath: /shared
```

A sidecar is just a second entry in the same `containers` list — both containers mount the shared `emptyDir` volume.

DO IT — TWO WAYS

`kubectl run` gets you a skeleton, not a multi-container Pod

IMPERATIVE — A STARTING POINT	DECLARATIVE — THE ONLY WAY TO ADD MORE
`kubectl run web --image=nginx:1.25`	`kubectl apply -f pod-sidecar.yaml`
Gives you exactly one container, no flag adds more	Hand-write `initContainers:` / a second `containers:` entry + shared volumes

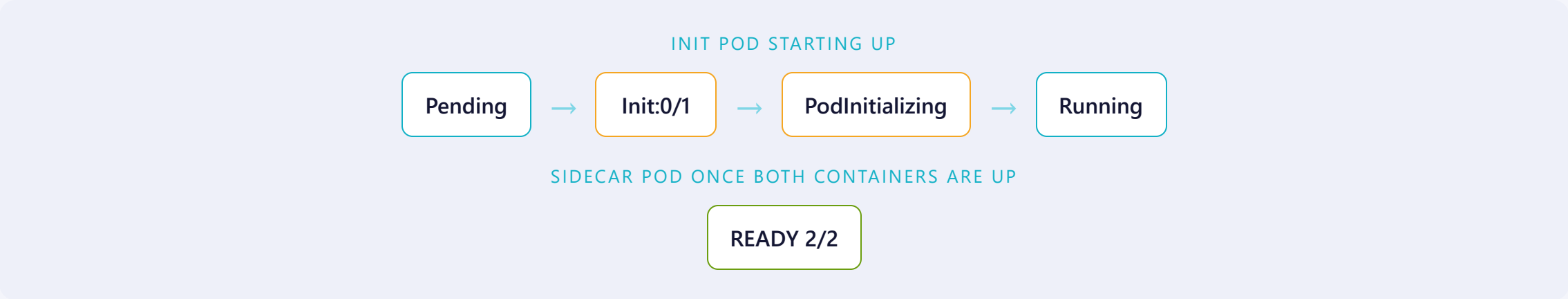
THE BRIDGE TRICK

Generate a single-container skeleton, then hand-edit in the second container and the shared volume:

```
$ kubectl run web --image=nginx:1.25 \
    --dry-run=client -o yaml > pod.yaml  # then add initContainers: / containers: by hand
```

SEE IT

The status trail proves the ordering



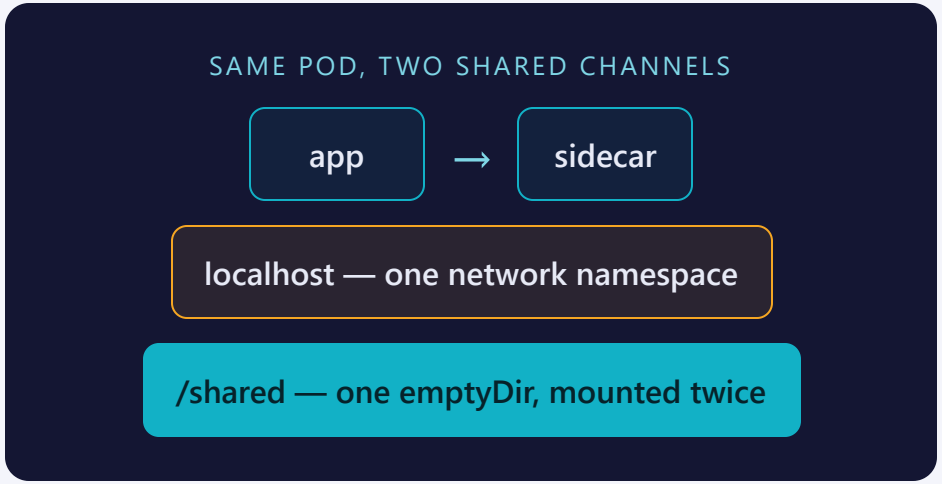
`kubectl describe pod` shows the init container's `Pulled → Created → Started` trail complete *before* the app container's begins — the ordering isn't just documented, it's in the event log.

THE PUNCHLINE

They share a fabric, not a filesystem

The sidecar never reads the app's process or its image — it only ever sees what lands in the shared `emptyDir`. Each container keeps its own root filesystem.

Same story on the network side: both containers answer on the same Pod IP, so one can reach the other over `localhost` — no Service, no DNS, needed inside the Pod.



GOTCHAS — THE ONES THAT BITE

Where people trip

- **Forgetting `-c NAME`.** With more than one container, `logs` and `exec` need to know which one.
- **Assuming the filesystem is shared.** Only mounted volumes are — each container still has its own root FS.
- **A crashing init container blocks the Pod forever.** The app never starts until every init container exits `0`.
- **READY doesn't count init containers.** A stuck one shows as `Init:0/1` in STATUS, not in READY.

YOUR MULTI-CONTAINER TOOLKIT

Commands to keep at hand

```
$ kubectl get pod NAME                        # READY x/y, STATUS Init:N/M or Running
$ kubectl logs NAME -c CONTAINER              # one container's stdout/stderr
$ kubectl exec -it NAME -c CONTAINER -- sh    # shell into one container
$ kubectl describe pod NAME                  # ordered Events: init done, then app started
$ kubectl get pod NAME -o jsonpath='{.status.containerStatuses[*].ready}'  # per-container ready
```

Tip: `Init:N/M` in STATUS means still in the init phase — check that init container's logs first.

RECAP

Multi-container Pods in one breath

- A Pod can hold multiple containers sharing network & volumes — never the filesystem
- Sidecars run alongside the app for its whole life; init containers run first, in order, to completion
- Target a container explicitly with `-c` for `logs`/`exec`
- `kubectl run` can't build these — hand-edit the declarative YAML

HANDS-ON

Now run `Lab 2.2`

`labs/2.2/`

Next: **2.3 — Deployments & ReplicaSets**